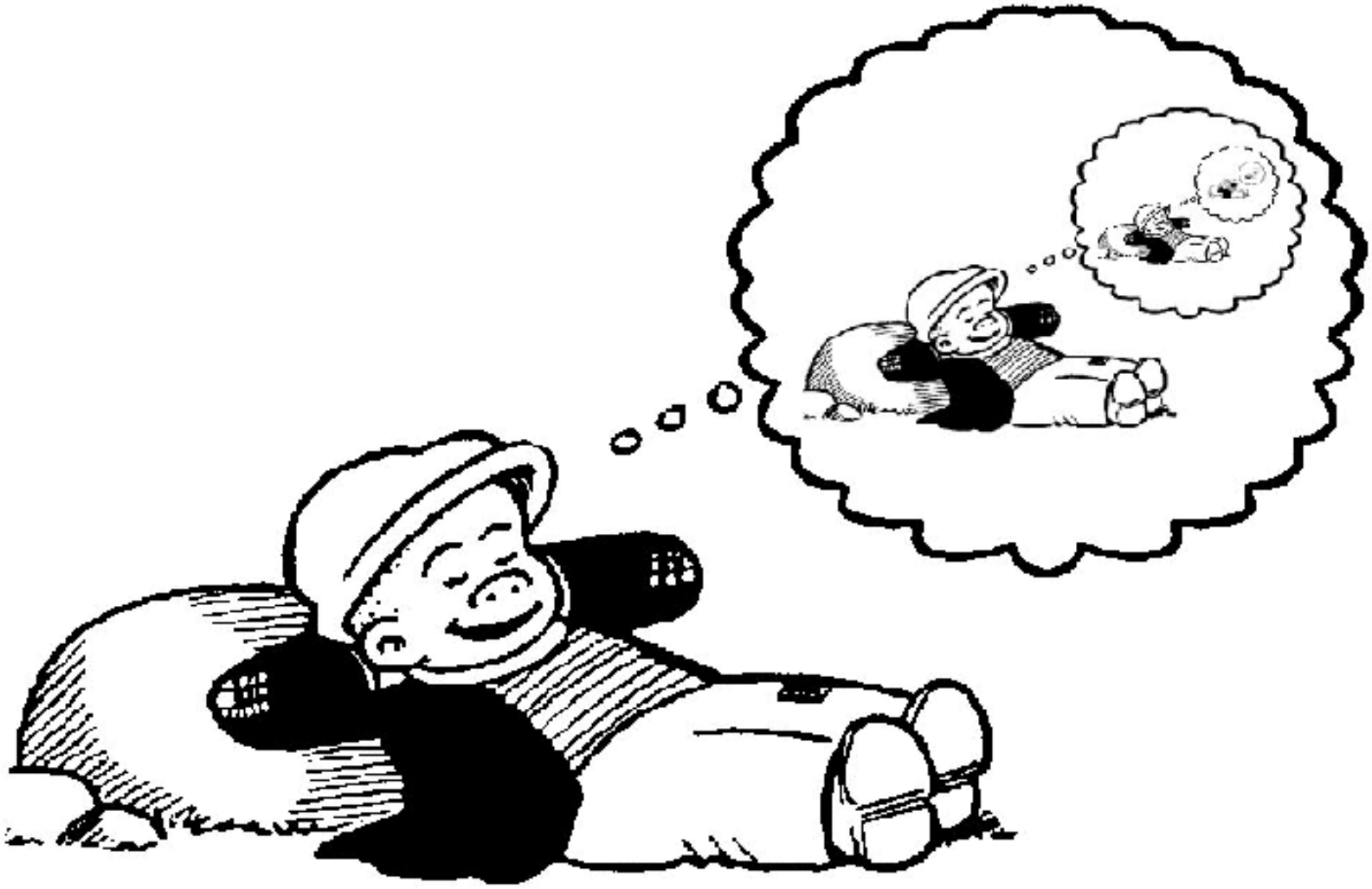
CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

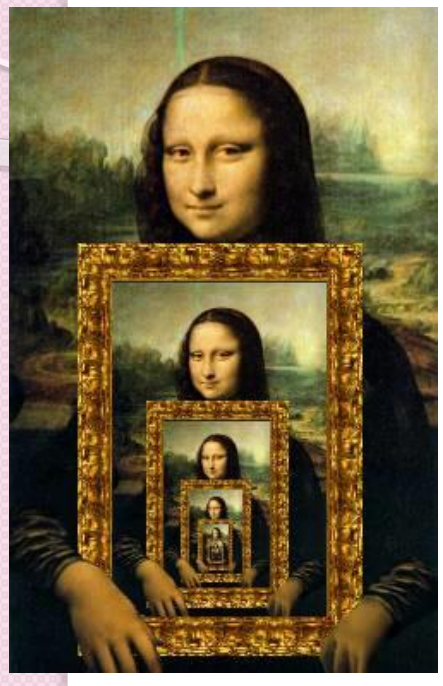
Chương 2

GIẢI THUẬT ĐỆ QUY
(Recursive Algorithms)

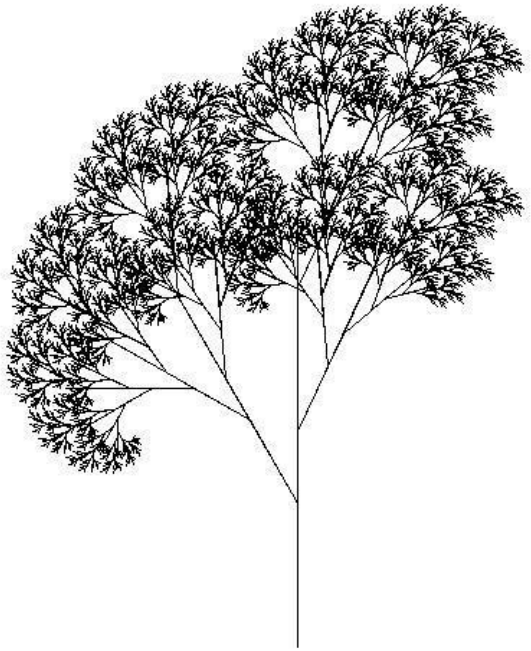
Đệ quy



Đệ quy (recursion)







- **Đối tượng đệ quy**: đối tượng bao gồm chính nó hoặc được định nghĩa dưới dạng của chính nó
- **Giải thuật đệ quy:** trong các bước thực hiện giải thuật có ít nhất một lần gọi lại chính nó

Tính giai thừa

- 4! = 4 * 3 * 2 * 1
- Tổng quát:

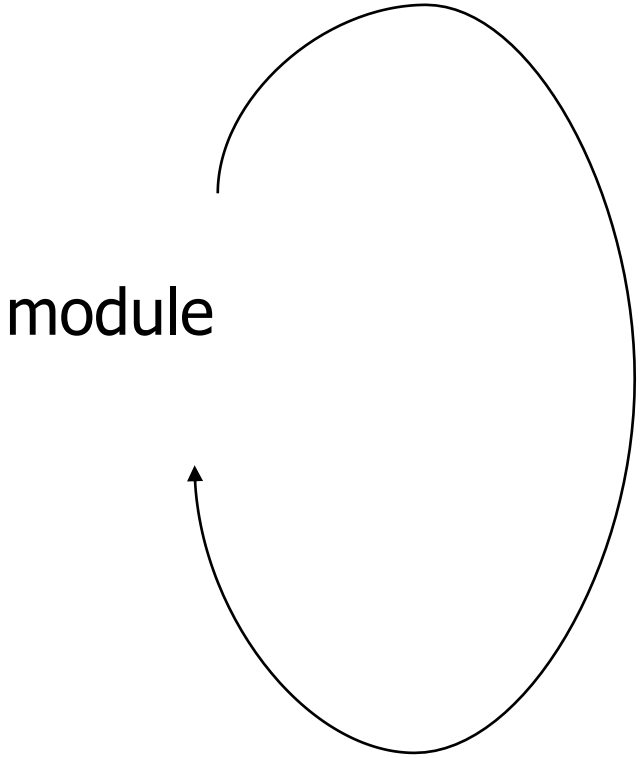
n! = n * (n-1) * (n-2) * … * 1

= n* (n-1)!

Hay

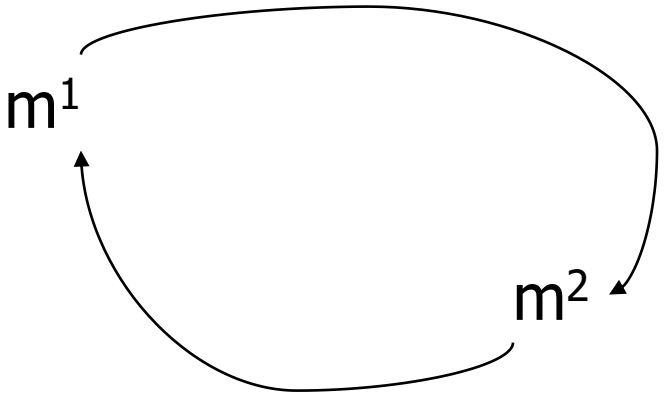
$$n! = \begin{cases} 1 & ,nếu\ n = 1 \\ n * (n-1)! & ,nếu\ n > 1 \end{cases}$$

Đệ quy trực tiếp (Direct Call)

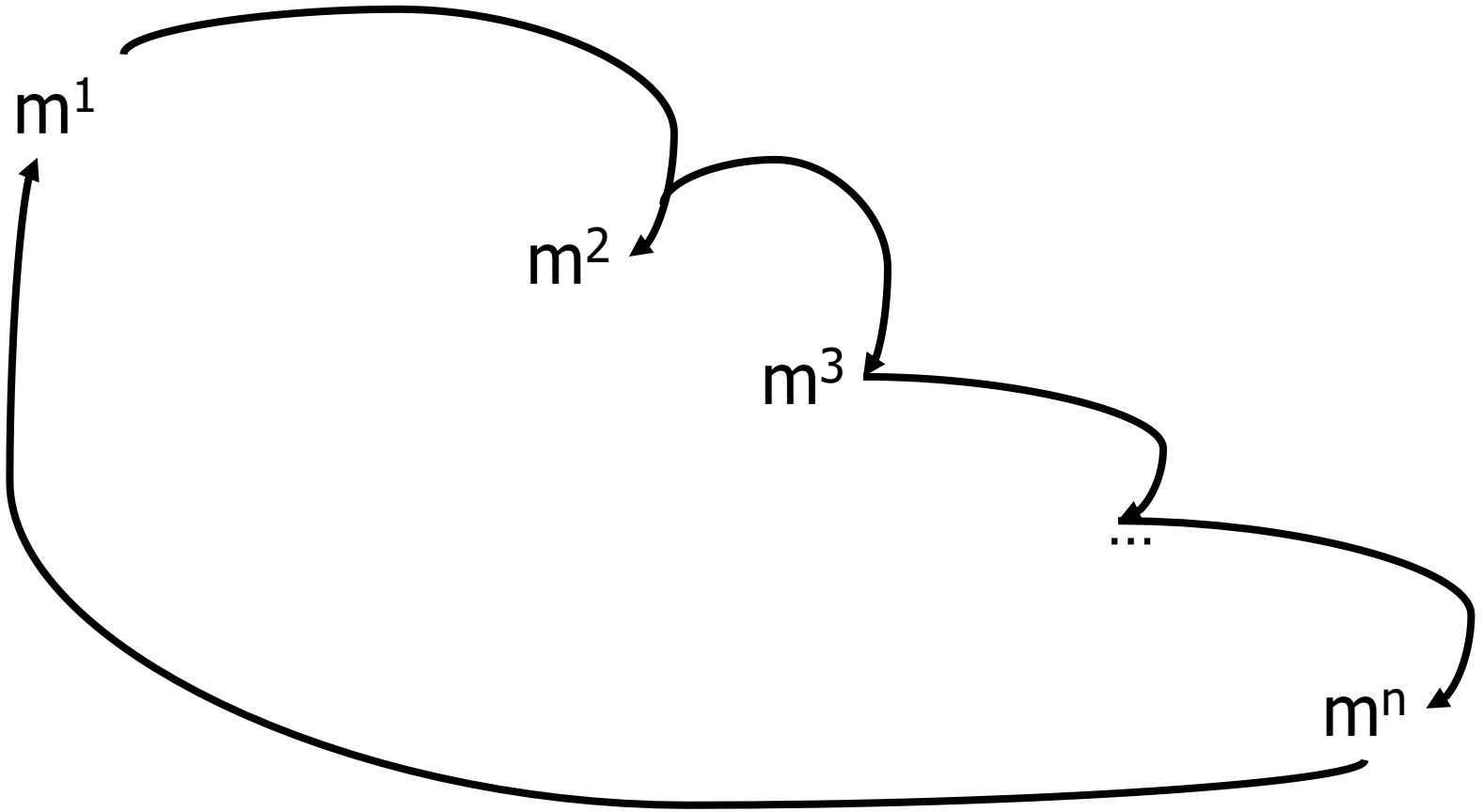



```
procedure proc;
begin
        :
    proc ();
        :
end;
```

Đệ quy gián tiếp (Indirect Call)



Đệ quy gián tiếp (Indirect Call)



Indirect Call (2)

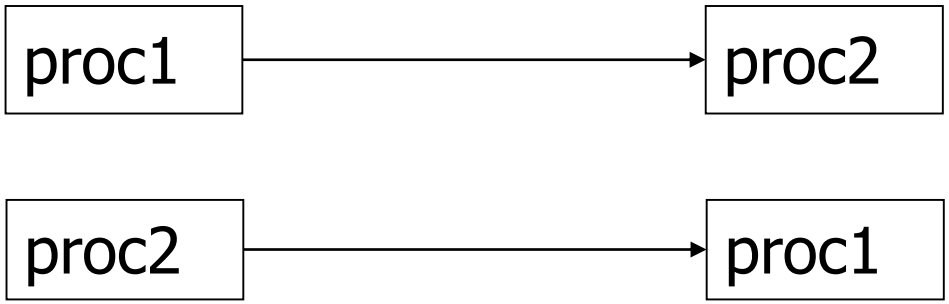
```
procedure proc1;
begin
   :
  proc2;
end;

procedure proc2;
begin
   :
  proc3;
end;

procedure proc3;
begin
   :
  proc1;
end;
```

An Issue With Indirect Recursion

Ví dụ:

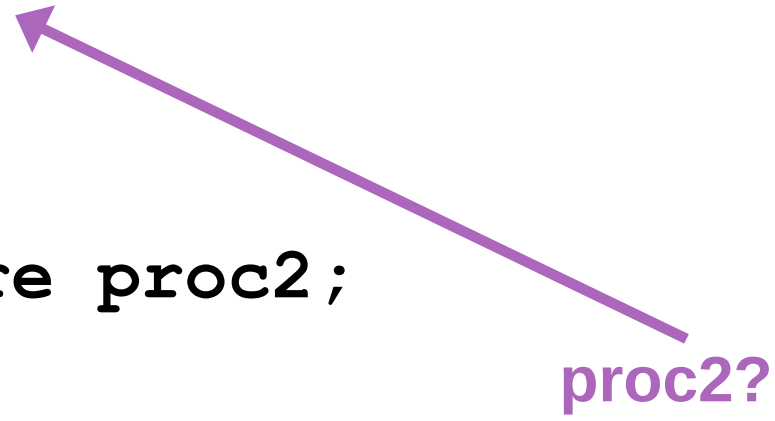


Nên định nghĩa thủ tục nào trước?

Procedure Proc1 trước?

```
procedure proc1;
begin
  :
  proc2;
  :
end;

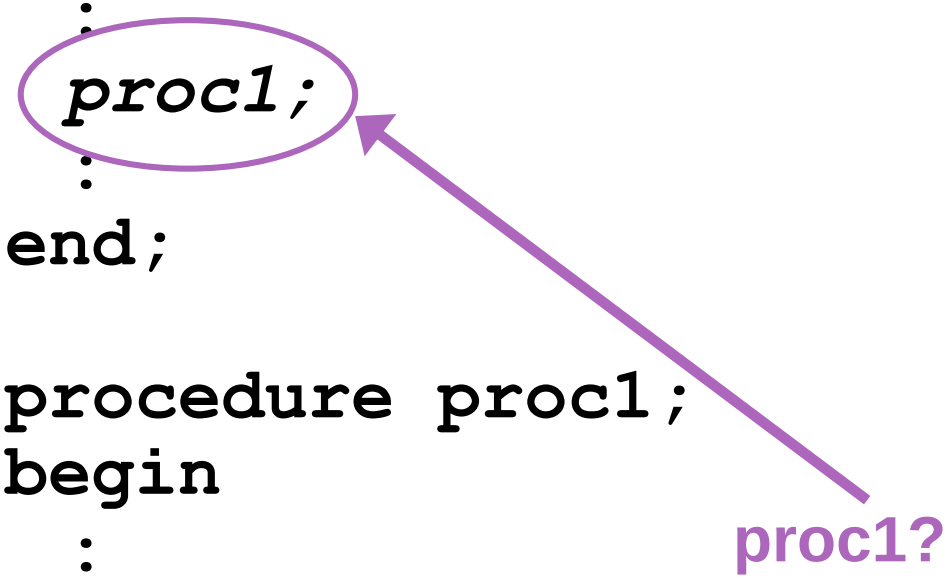
procedure proc2;
begin
  :
  proc1;
  :
end;
```



Procedure Proc2 trước?

```
procedure proc2;
begin
  :
  proc1;
  :
end;

procedure proc1;
begin
  :
  proc2;
  :
end;
```



Giải pháp

Ví dụ

```
procedure proc1;
begin
   :
   proc2;
   :
end;

procedure proc2;
begin
   :
   proc1;
   :
end;
```

Giải pháp

Ví dụ

```
procedure proc2;
FORWARD;(trong Pascal)

procedure proc1;
begin
   :
   proc2;
   :
end;

procedure proc2;
begin
   :
   proc1;
   :
end;
```

- **Trong C?**
- Sử dung forward declaration: dùng câu lệnh khai báo gọi là nguyên mẫu hàm (function prototype)
- Nguyên mẫu hàm gồm: kiểu trả về của hàm,tên hàm, tham số, nhưng không có thân hàm, kết thúc bằng dấu ";"

```
double tong(double x, double y);
double tich(double x, double y);
int main() {
   double x = 5., y = 10.;
   tong(x, y);
   tich(x, y);
   return 0;
}
double tong(double x, double y) { return x+y; }
double tich(double x, double y) { return x*y; }
```

Giải pháp

Ví dụ

```
procedure proc2;
FORWARD;(trong Pascal)

procedure proc1;
begin
   :
   proc2;
   :
end;

procedure proc2;
begin
   :
   proc1;
   :
end;
```

Trong Python?

Không cần làm gì cả!

```
1  def ping(i):
2      if i>0:
3          return pong(i-1)
4      return "0"
5  def pong(i):
6      if i>0:
7          return ping(i-1)
8      return "1"
9  print(ping(4))
```

Yêu cầu đối với giải thuật đệ quy

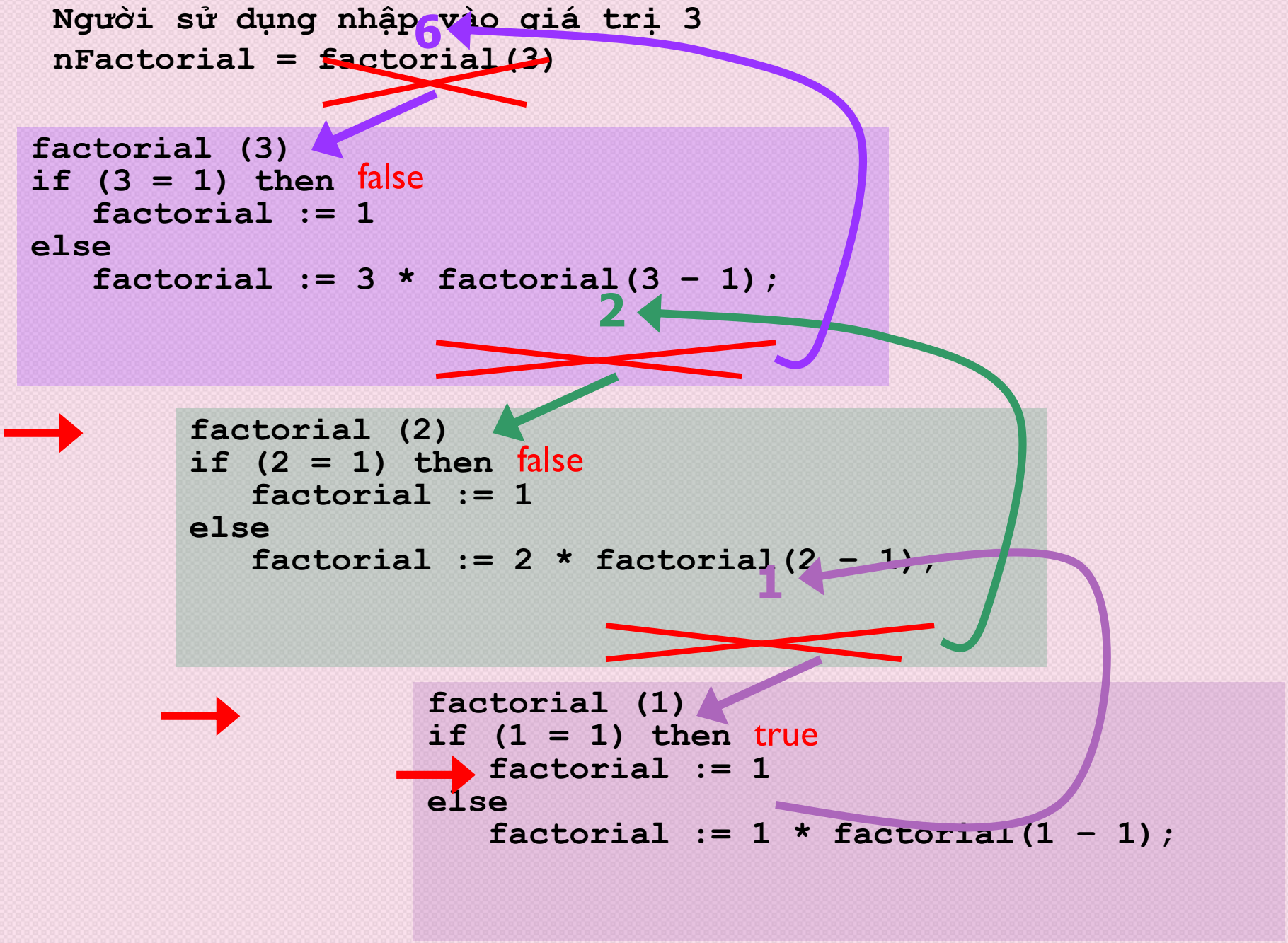
1) Có trường hợp suy biến

2) Lời gọi đệ quy

Ví dụ- Tính giai thừa

```
function factorial (num :integer):integer;
begin
    if (num = 1) then
        factorial := 1
    else
        factorial := num*factorial(num - 1);
end;

BEGIN
  nFactorial := factorial(number);
END
```


Người sử dụng nhập vào giá trị 3
nFactorial = factorial(3)
6
factorial (3)
if (3 = 1) then false
factorial := 1
else
factorial := 3 * factorial(3 - 1);
2
factorial (2)
if (2 = 1) then false
factorial := 1
else
factorial := 2 * factorial(2 - 1);
1
factorial (1)
if (1 = 1) then true
factorial := 1
else
factorial := 1 * factorial(1 - 1);

Sử dụng đệ quy khi nào

- Khi bài toán có thể chia thành các bài toán con nhỏ hơn
- Kết quả của bước này có thể được dùng cho bước trước.
- Có trường hợp suy biến.
- Tập hợp các kết quả lại có thể giải được bài toán đã cho

Quy tắc viết thuật toán đệ quy

1. Xác định trường hợp suy biến
2. Xác định trường hợp tổng quát.
3. Kết hợp trường hợp suy biến và tổng quát thành thuật toán

Kết hợp trường hợp suy biến và tổng quát thành thuật toán

- Mỗi lời gọi phải giảm được kích thước bài toán và tiến dần đến trường hợp suy biến.
- Khi đạt tới trường hợp suy biến thì thuật toán phải kết thúc mà không có thêm lời gọi đệ quy nào→ nhận giá trị trả về

Nhược điểm của pp đệ quy

- Bộ nhớ?
- Thời gian?

Một số lỗi thường gặp

- Không có trường hợp suy biến
- Không có bước dẫn tới trường hợp suy biến
- Sử dụng quá nhiều tài nguyên (e.g., variable declarations) cho mỗi lời gọi hàm

Không có trường hợp suy biến

```
function factorial (num : integer): integer;
begin
   factorial := num * factorial (num - 1);
end;
```

Không có trường hợp suy biến

```
function factorial (num : integer): integer;
begin
    factorial := num * factorial (num - 1);
end;
```



Dừng khi nào???

Không có bước dẫn tới trường hợp suy biến

```
function factorial (num : integer): integer;
begin
   if (num = 1) then
         factorial := 1
   else
         factorial := num * factorial (num -1);
end;
```



Dẫn tới trường hợp suy biến

Sử dụng quá nhiều tài nguyên

```
procedure proc;
var
   arr : array [1..1000000] of char;
begin
   proc;
end;
```

Khử đệ quy

- Sử dụng vòng lặp

Ví dụ: Tìm UCLN

UCLN (Greatest Common Divisor):

$$\text{UCLN} = \begin{cases} a & ,\text{nếu } b = 0 \\ b & ,\text{nếu } a = 0 \\ UCLN(b, a \bmod b), & \text{trong các trường hợp khác} \end{cases}$$

Pseudocode

Euclidean Algorithm for Greatest Common Divisor

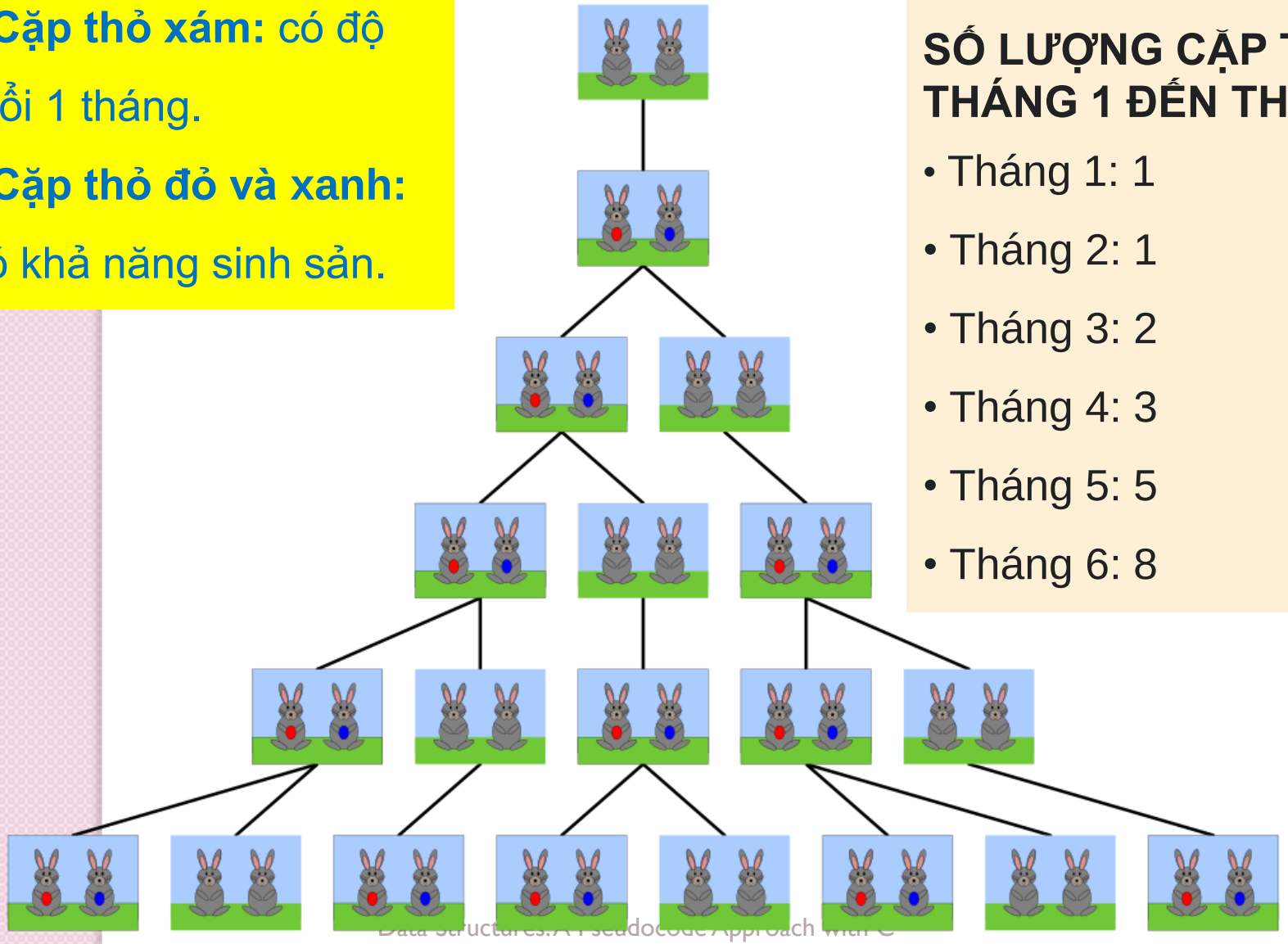
```
Algorithm gcd (a, b)
Calculates greatest common divisor using the Euclidean algo-
rithm.
   Pre  a and b are positive integers greater than 0
   Post greatest common divisor returned
1  if (b equals 0)
   1   return a
2  end if
3  if (a equals 0)
   2   return b
4  end if
5  return gcd (b, a mod b)
end gcd
```

Fibonacci numbers

- Một đôi thỏ (gồm một thỏ đực và một thỏ cái) không sinh cho đến khi chúng đủ 2 tháng tuổi.
- Sau khi đủ 2 tháng tuổi, mỗi tháng, mỗi đôi thỏ sinh một đôi thỏ con (gồm một thỏ đực và một thỏ cái).
- Hỏi sau n tháng có bao nhiêu đôi thỏ, nếu đầu năm (tháng Giêng) có một đôi thỏ sơ sinh

Fibonacci numbers

- **Cặp thỏ xám:** có độ tuổi 1 tháng.
- **Cặp thỏ đỏ và xanh:** có khả năng sinh sản.



SỐ LƯỢNG CẶP THỎ TỪ THÁNG 1 ĐẾN THÁNG 6:

- Tháng 1: 1
- Tháng 2: 1
- Tháng 3: 2
- Tháng 4: 3
- Tháng 5: 5
- Tháng 6: 8

Fibonacci numbers

- Mỗi số là tổng của 2 số liền trước.
- Dãy số Fibonacci:
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …

$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) & \text{otherwise} \end{cases}$$

Fibonacci Numbers Recursive Definition